

Apache James メールサーバ構築手順書 (Windows・ApacheDS 連携)

第 0.5 版

2026-07-14

0. 改訂履歴

版	日付	変更内容	備考
0.1	2026-07-13	初版。目的・構成・パラメータ・ロードマップ、フェーズ 1～5 (導入/LDAP 連携/プロトコル設定/起動・サービス化/動作確認・PLM 連携)、トラブルシュート、付録を記載	未検証 (要実機確認)。実機で確認しながら改訂する
0.2	2026-07-13	\$7.4 を拡充：外部ドメイン宛の送信を完全に禁止する設定 (RemoteDelivery を削除し Bounce で送信失敗にする) を追加。踏み台 (オープンリレー) リスクを原理的に無くせる旨も記載。\$9.2 にユーザー間 (01PLM01→01PLM02) のテストメール送受信手順 (Thunderbird 2 アカウント) と外部宛が送信失敗になることの確認を追加。\$10 にメールソフト設定のトラブルを追加	外部送信禁止・クライアント検証
0.5	2026-07-14	\$8.4 に実機のサービス表示名「 Apache James :: Server :: Spring :: App 」を明記。DisplayName に記号を含むため操作は Name を用いる旨を追記。フェーズ 1～5 + サービス化まで実機で完走 (本書は実機検証済みとなった)	サービス名の確定・完走
0.4	2026-07-14	\$8.4 「Windows サービス化」を実行手順に詳細化：james.bat install/start/stop/remove (Tanuki 同梱・NSSM 不要)、スタートアップ「自動」設定、再起動堅牢性の確認、ApacheDS への	サービス化手順の詳細化

版	日付	変更内容	備考
0.3	2026-07-14	依存関係設定、設定変更時の <code>restart</code> を記載 実機検証で完走（フェーズ 1～5）。判明した多数の知見を反映：\$5.1（James 3.9 は Java 21 必須-SSO 環境の Java 17 では動かず、3.8.2 を採用）／\$5.4（*-template.xml は無い。.xml を直接編集。バックアップは <code>.xml.org</code>）／\$6.1（タグ名は supportsVirtualHosting。enableVirtualHosting は無視され Unknown user の原因に）／\$7.1（verifyIdentity=false が必須）／\$7.3（暗号化なしでは plainAuthDisallowed=false が必須。無いと IMAP でパスワード入力画面すら出ず接続断）／\$7.4（外部送信拒否は 2 層構造 = SMTP 即拒否 + Bounce。案 C 採用 = 第 1 層で拒否・<notice> で規則違反を明示・PLM 側で応答コード判定。RemoteAddrNotInNetwork を authorizedAddresses と揃える）／\$8（起動 <code>run.bat</code>／サービス化 <code>james.bat</code> = Tanuki 同梱・NSSM 不要）／\$9（Send-MailMessage・IMAP 直接接続での切り分け／メールアドレスは小文字で扱う理由）／\$10（無害ログ・Unknown user・plainAuth・大文字小文字の切り分け）／付録（メール小文字と LDAP/SSO 大文字の使い分け）	実機検証版

本書の位置づけ：本書は **James 3.8.2** を用いて **Windows** 実機でフェーズ **1**～**5** + サービス化まで通して検証済み（v0.5 時点）。設定値・ファイル名・挙動は実機の結果を反映している。環境（版・ネットワーク・LDAP 構成）が異なる場合は、該当箇所を読み替えること。

1. 目的・前提・スコープ

1.1 目的

社内テスト環境の PLM システム（Web アプリサーバ・バッチサーバ）がメール送信の動作確認を行えるようにする。社内メールサーバへのネットワークアクセスが制限されたため、**Web** アプリサーバ自身にメールサーバの役割を持たせる。

同サーバには SSO 検証用に **ApacheDS**（**LDAP**）を導入済みであるため、**Apache James** のユーザー情報を **ApacheDS** と共有し、二重管理を避ける。

1.2 要件

#	要件	本書での実現方法
1	Web アプリサーバ・バッチサーバからの SMTP 送信要求を受け付ける	James の SMTP（25/tcp）。両サーバの IP からのリレーを許可
2	ネットワーク上のクライアントからの IMAP/POP3 要求を受け付ける	James の IMAP（143/tcp）／POP3（110/tcp）
3	暗号化は不要（テスト環境・確認用途のみ）	STARTTLS／SSL は無効。平文で運用（IMAP は plainAuth Disallowed=false が必要）
4	ユーザーは ApacheDS と共有する	James の ReadOnlyUsersLDAPRepository で LDAP を参照
5	外部ドメイン宛の送信は禁止（自ドメイン内で完結）	2 層で防御：SMTP リレー拒否（第 1 層・即エラー）＋ RemoteDelivery 不在による Bounce（第 2 層）。\$7.4

1.3 スコープ外

- ・ 暗号化（STARTTLS／SMTPS／IMAPS／POP3S）。テスト環境のため対象外。
- ・ 外部インターネットへのメール送信（リレー）。社内テスト環境内で完結させる（PLM が送ったメールを、同じ James 上のメールボックスで受信して確認する）。
- ・ スパム対策・ウイルス対策・大規模運用のためのチューニング。
- ・ 本番運用（可用性・バックアップ・監査）。

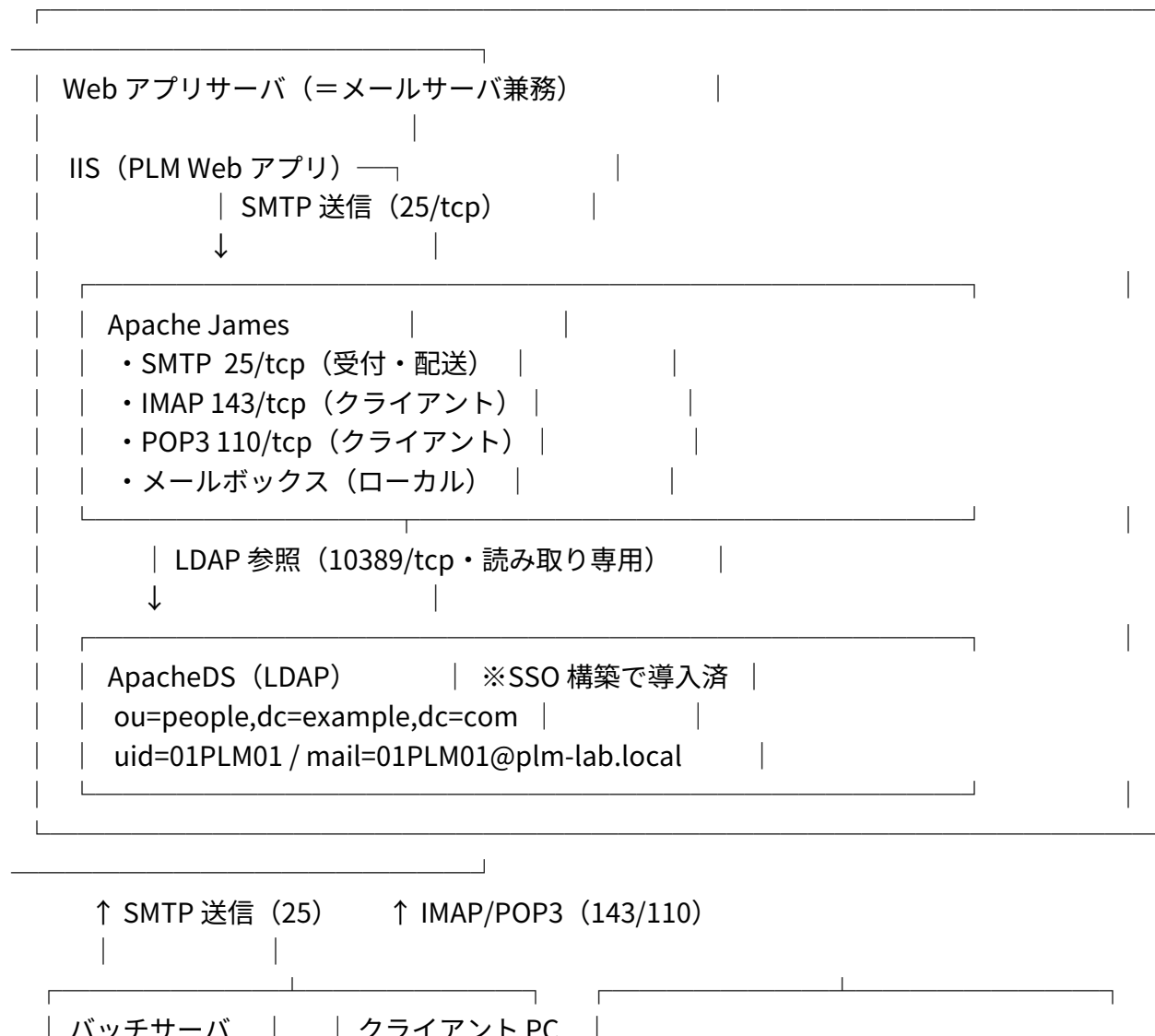
1.4 前提

- ・ **ApacheDS** が構築済みであること（Shibboleth SSO 構築手順書のフェーズ 3 完了状態）。
 - LDAP : ldap://localhost:10389

- baseDN : dc=example,dc=com、ユーザー : ou=people,dc=example,dc=com
- ユーザー : uid=01PLM01 (mail=01PLM01@plm-lab.local) 、uid=01PLM02 (同様)
- 検索用アカウント : uid=idp-reader,ou=people,dc=example,dc=com / idp-reader
- **Java 17 (Temurin)** が導入済み (JAVA_HOME=C:\opt\jdk-17) 。James も Java で動作する。
- 導入方針は SSO 手順書と同じ : **zip** を **C:\opt** 配下に展開 (空白を含まないパス) 。

2. 構成

2.1 全体構成



(PLM バッチ)	(メールソフト)
-----------	----------

2.2 設計上の要点

- ユーザーは **LDAP** と共有：James は ApacheDS を読み取り専用（ReadOnlyUsersLDAPRepository）で参照する。**James** 側でユーザーを作成・削除しない（ユーザー管理は ApacheDS 側で行う。SSO 環境と同じユーザーがそのままメールアドレスになる）。
- メールアドレス = **LDAP** の **mail** 属性：01PLM01@plm-lab.local。SSO の NameID（emailAddress 形式）と同一の値であり、**PLM** が「誰に送るか」を決める際の識別子と一致する。
- ドメインは **plm-lab.local**：James にこのドメインを「ローカルドメイン」として登録することで、宛先が @plm-lab.local のメールは外部に出さず、**James** 内のメールボックスに配送される（テスト環境内で完結）。
- 認証・暗号化なし：PLM からの SMTP 送信は認証なしで行い、送信元 IP でリレーを許可する（authorizedAddresses）。テスト環境限定の割り切り。
- 外部ドメイン宛の送信は禁止（2 層防御・案 C）：① SMTP のリレー制御で、許可外の送信元からの外部宛をその場で拒否（5.7.1 relaying denied／第 1 層・同期エラー）、② mailletcontainer.xml に RemoteDelivery が存在しないため、受理された外部宛も **Bounce**（第 2 層）で差出人に返る。IP リレー許可があっても外部への踏み台（オープンリレー）になり得ない（\$7.4）。
- メールアドレスは小文字で扱う：James はメールアドレス（ユーザー名）を内部で小文字に正規化する（james-cli listusers は 01plm01@plm-lab.local と表示）。LDAP の mail／SSO の REMOTE_USER は大文字（01PLM01@...）のままで、メール送信の宛先・**IMAP** ログイン ID は小文字を用いる。LDAP の mail は SSO のため大文字を維持し、**PLM** 側はメール宛先を組み立てる際に小文字化（**To Lower()**）する（\$9・付録 B）。

3. パラメータ・アカウント

3.1 パラメータ

項目	値	備考
James インストール先	C:\opt\james	zip 展開 + リネーム
メールドメイン	plm-lab.local	LDAP の mail 属性のドメインと一致させる
SMTP ポート	25/tcp	PLM Web／バッチからの送信受付
POP3 ポート	110/tcp	クライアント受信
IMAP ポート	143/tcp	クライアント受信
LDAP 接続先	ldap://localhost:103	ApacheDS（同一サーバ）

項目	値	備考
	89	
LDAP userBase	ou=people,dc=example,dc=com	
LDAP 検索アカウント	uid=idp-reader,ou=people,dc=example,dc=com / idp-reader	SSO 構築で作成済のものを流用
ユーザー識別属性	mail (例 01PLM01@plm-lab.local)	メールアドレスでログインする構成
暗号化	なし (平文)	テスト環境のため

3.2 テスト用アカウント (ApacheDS に既存)

ユーザー	メールアドレス (= ログイン ID)	パスワード
01PLM01	01PLM01@plm-lab.local	01PLM01
01PLM02	01PLM02@plm-lab.local	01PLM02

パスワードは ApacheDS に登録済みのもの (SSO 構築時に LDIF で投入)。James は認証時に LDAP へ bind して検証するため、**James** 側にパスワードを持たせない。

4. ロードマップ

フェーズ	内容	状態
1	James の入手・展開・Java 確認	☐ 未実施
2	LDAP 連携 (usersrepository.xml) とドメイン登録	☐ 未実施
3	プロトコル設定 (SMTP/POP3/IMAP・暗号化なし・リレー許可)	☐ 未実施
4	起動・サービス化・ファイアウォール	☐ 未実施
5	動作確認 (telnet/メールソフト/PLMからの送信)	☐ 未実施

5. フェーズ 1 : James の入手・展開

5.1 版の選定 (重要 : Java のバージョンに注意)

- Apache James の公式ダウンロードページ (<https://james.apache.org/download.cgi>) から、**Spring 版 (james-server-spring-app-*.zip)** を入手する。
 - Spring 版は conf/ 配下の XML を編集する従来型の構成で、単体サーバ・ファイル/JPA ストレージ・LDAP 連携という本書の用途に最も適する。
 - Guice 版/Distributed 版は Cassandra 等の外部ミドルウェアを前提とする構成があり、本用途には過剰。
-  **【最重要】** 版と Java の対応 : **James 3.9.0** は **Java 21** が必須であり、**Java 17** では起動しない (下記エラー)。本環境は SSO (Shibboleth IdP) の要件で **Java 17** を使うため、**James** は **3.8.2** を採用する (Java 17 で動作)。
 - 実機で使用 : **james-server-spring-app-3.8.2-app.zip**。
 - 3.9.0 を Java 17 で起動したときのエラー :

```
java.lang.UnsupportedClassVersionError: ... has been compiled by a more recent version of
the Java Runtime (class file version 65.0), this version of the Java Runtime only
recognizes class file versions up to 61.0

(65.0 = Java 21、61.0 = Java 17)
```
 - Java 21 を別途導入して James だけそちらで動かす選択肢もあるが、JAVA_HOME の管理が複雑になるため、同一サーバでは **3.8.2** を使うのが確実。

5.2 展開

例 : C:\lab\installers に保存した zip を C:\opt に展開

```
Expand-Archive -Path "C:\lab\installers\james-server-spring-app-3.8.2-app.zip" -DestinationPath "C:\opt" -Force
```

展開されたフォルダ名を確認

```
Get-ChildItem C:\opt | Where-Object Name -like "james*"
```

C:\opt\james にリネーム (パスを短く・空白なしに)

```
Rename-Item "C:\opt\james-server-spring-app-3.8.2" "C:\opt\james"
```

構成を確認 (conf/bin/lib などがあること)

```
Get-ChildItem C:\opt\james
```

5.3 Java の確認

```
echo $env:JAVA_HOME    # C:\opt\jdk-17
```

```
java -version          # 17.x
```

5.4 設定ファイルの準備 (3.8.2 はテンプレート方式ではない)

3.8.2 では ***-template.xml** は存在せず、**conf** 直下に **.xml** が最初から配置されている。したがって、テンプレートをコピーする作業は不要で、既存の **.xml** を直接編集する。

```
cd C:\opt\james\conf
Get-ChildItem *-template.xml # → 何も返らない (テンプレートは無い)
Get-ChildItem *.xml          # → usersrepository.xml, domainlist.xml, smtpserver.xml,
                             # pop3server.xml, imapserver.xml, mailetcontainer.xml など
```

編集前にバックアップを取る場合は、拡張子を変えて取得する (.xml.org 等)。

```
Copy-Item usersrepository.xml usersrepository.xml.org -Force
Copy-Item domainlist.xml      domainlist.xml.org      -Force
Copy-Item smtpserver.xml      smtpserver.xml.org      -Force
Copy-Item imapserver.xml      imapserver.xml.org      -Force
Copy-Item mailetcontainer.xml mailetcontainer.xml.org -Force
```

⚠ バックアップの拡張子に注意 (SSO 手順書での教訓) : conf 配下に **.xml** のままコピーを置かない (James が読み込んで競合し得る)。**.xml.org** のように拡張子を変えるか、conf の外 (C:\lab\james-backup\) に置く。**.org** なら James は読まない。

本書で編集するファイル : usersrepository.xml (LDAP 連携)、domainlist.xml (ドメイン)、smtpserver.xml (SMTP)、imapserver.xml (IMAP)、pop3server.xml (POP3・本書では無効化)、mailetcontainer.xml (配送・外部送信禁止)。

6. フェーズ 2 : LDAP 連携とドメイン登録

6.1 ユーザーリポジトリを LDAP に切り替える (usersrepository.xml)

C:\opt\james\conf\usersrepository.xml を編集し、既定の **JPA/File** ベースのリポジトリをコメントアウトし、**LDAP** リポジトリを有効化する。

```
<usersrepository name="LocalUsers"
  class="org.apache.james.user.ldap.ReadOnlyUsersLDAPRepository"
  ldapHost="ldap://localhost:10389"
  principal="uid=idp-reader,ou=people,dc=example,dc=com"
  credentials="idp-reader"
  userBase="ou=people,dc=example,dc=com"
  userIdAttribute="mail"
  userObjectClass="inetOrgPerson">
<supportsVirtualHosting>true</supportsVirtualHosting>
```



```
<enableVirtualHosting>true</enableVirtualHosting>
</usersrepository>
```

⚠️【最重要】バーチャルホスティングのタグ名は **supportsVirtualHosting** : 3.8.2 の ReadOnlyUsersLDAPRepository が認識するタグは **supportsVirtualHosting** であり、**enableVirtualHosting** は認識されず無視される。認識されないとバーチャルホスティングが無効（既定）になり、James は宛先を「ローカルパートのみ」（01plm01）に変換して照合するため、mail=01PLM01@plm-lab.local と一致せず **5.1.1 Unknown user** で拒否される。両方書いておくと版差に対応できる（認識されないほうは無視される）。

症状の分かりにくさ：この状態でも james-cli listusers には 01plm01@plm-lab.local が表示される（LDAP は読めている）のに、SMTP の宛先照合では見つからない、という矛盾した状態になる。LDAP・ドメイン設定がすべて正しくても発生するため、原因にたどり着きにくい（\$10 の切り分け参照）。

各項目の意味：

属性	値	説明
class	ReadOnlyUsersLDAPRepository	読み取り専用。James からのユーザー作成・削除は無視される（ユーザー管理は Apache DS 側で行う）
ldapHost	ldap://localhost:10389	ApacheDS（同一サーバ・平文）
principal / credentials	uid=idp-reader,... / idp-reader	LDAP を検索するためのバインドアカウント（SSO 構築で作成済）
userBase	ou=people,dc=example,dc=com	ユーザーを探す起点
userIdAttribute	mail	ログイン ID として使う属性。mail（01PLM01@plm-lab.local）を使う
userObjectClass	inetOrgPerson	ユーザーエントリの objectClass
supportsVirtualHosting	true	ユーザー名がメールアドレス形式であることを示す。userIdAttribute="mail" と対で必要。 3.8.2 では enableVirtualHosting ではなくこのタグ名（下記の注記）

なぜ `userIdAttribute="mail"` なのか：メールサーバのユーザーは「メールアドレス」で識別されるのが自然であり、SSO の NameID (emailAddress 形式) ととも一致する。uid (01PLM01) を使う構成も可能だが、その場合は `enableVirtualHosting=false` とし、James が内部でドメインを補う形になる。本書はメールアドレスでログインする構成を採る。

認証の流れ：クライアントが 01PLM01@plm-lab.local / 01PLM01 で IMAP ログイン - James が LDAP で mail=01PLM01@plm-lab.local のエントリを検索 - そのエントリの DN で LDAP に **bind** して認証。パスワードは ApacheDS が保持しているものが使われる (James はパスワードを持たない)。

6.2 ローカルドメインの登録 (domainlist.xml)

宛先 @plm-lab.local のメールを外部に出さず、**James** 内で配送するために、このドメインをローカルドメインとして登録する。

```
<domainlist class="org.apache.james.domainlist.xml.XMLDomainList">
  <domainnames>
    <domainname>plm-lab.local</domainname>
    <domainname>localhost</domainname>
  </domainnames>
  <autodetect>>false</autodetect>
  <autodetectIP>>false</autodetectIP>
</domainlist>
```

autodetect を false にして、明示したドメインだけをローカル扱いにする (マシン名などが混ざるのを防ぐ)。

⚠ 実機確認事項：版によっては domainlist.xml のクラス指定や既定値が異なる。テンプレートの内容を確認し、plm-lab.local を追加する形にすること。**James CLI (james-cli adddomain plm-lab.local)** でドメインを追加する方式もあるので、テンプレートの構成に応じて選ぶ。

7. フェーズ 3：プロトコル設定 (SMTP / POP3 / IMAP)

暗号化は不要のため、いずれも `startTLS enable="false"` / SSL 無効とする。

7.1 SMTP (smtpserver.xml) — PLM からの送信受付

C:\opt\james\conf\smtpserver.xml の要点：

```
<smtpserver enabled="true">
  <jmxName>smtpserver-global</jmxName>
  <bind>0.0.0.0:25</bind>          <!-- 全インターフェースの 25 番で待ち受け -->
  <connectionBacklog>200</connectionBacklog>
```

```

<tls socketTLS="false" startTLS="false"/> <!-- 暗号化なし -->
<auth>
  <announce>never</announce>      <!-- SMTP AUTH を要求しない -->
  <requireSSL>false</requireSSL>
  <!-- 認証なしでリレーを許可する送信元 (PLM Web/バッチサーバ) -->
  <authorizedAddresses>127.0.0.0/8, 192.168.137.0/24</authorizedAddresses>
  <verifyIdentity>false</verifyIdentity> <!-- ★必須 (下記) -->
</auth>
<maxmessageSize>0</maxmessageSize>
<addressBracketsEnforcement>true</addressBracketsEnforcement>
...
</smtpserver>

```

要点：

- **<bind>0.0.0.0:25</bind>**：バッチサーバなど他ホストからの接続も受けるため、127.0.0.1ではなく 0.0.0.0（全インターフェース）で待ち受ける。
- **<authorizedAddresses>**：ここに書いた IP/ネットワークからは、SMTP AUTH なしでリレーが許可される。PLM Web アプリサーバ（＝自ホスト）とバッチサーバの IP/サブネットを指定する。
 - 例：127.0.0.0/8（自ホスト）＋ 192.168.137.0/24（社内テストのサブネット）。実際の IP に合わせて設定すること。
 - 注意：ここに広いネットワークを書くと、その範囲の誰でも認証なしでメールを送れる。テスト環境のクローズドなネットワークに限定すること。
- **<verifyIdentity>false</verifyIdentity>**（必須）：これは「MAIL FROM のアドレスが、SMTP AUTH で認証したユーザーと一致するか検証する」設定。既定は true。本書は認証なし運用のため、true のままだと差出人が @plm-lab.local のメールが拒否される。必ず false にする。
- 暗号化・認証は無効（テスト用途）。

authorizedAddresses の記述形式：カンマ区切り・CIDR 表記（127.0.0.0/8, 192.168.137.0/24）。実際のネットワークに合わせる。ここに書いた IP からは外部宛リレーが「許可」される点に注意（\$7.4 で外部宛は結局 Bounce されるが、リレー許可の範囲は最小限にする）。

7.2 POP3 (pop3server.xml)

```

<pop3server enabled="true">
  <jmxName>pop3server</jmxName>
  <bind>0.0.0.0:110</bind>
  <tls socketTLS="false" startTLS="false"/>
  ...
</pop3server>

```

7.3 IMAP (imapserver.xml)

```
<imapserver enabled="true">
  <jmxName>imapserver</jmxName>
  <bind>0.0.0.0:143</bind>
  <tls socketTLS="false" startTLS="false"/>
  <!-- ★必須：暗号化なしでも平文認証を許可する（下記） -->
  <plainAuthDisallowed>false</plainAuthDisallowed>
  <plainAuthEnabled>true</plainAuthEnabled>
  ...
</imapserver>
```

クライアント PC から接続するため、いずれも **0.0.0.0**（全インターフェース）で待ち受ける。

⚠️【重要】暗号化なしでは **plainAuthDisallowed=false** が必須：James は既定で、暗号化されていない接続での平文ログインを拒否する（セキュリティ既定）。TLS を無効にした本構成でこれを許可しないと、**IMAP ログインが NO LOGIN failed. Plain login / authentication are disabled.** で失敗する。- 症状の分かりにくさ：メールソフト（Thunderbird 等）は、パスワード入力画面すら出さずに接続を切る（サーバが平文認証を許可していないと広告するため）。James のログにも Connection established → 数十 ms 後に Connection closed とだけ残り、ログイン試行のログが出ない。原因にたどり着きにくい。- 対処：<plainAuthDisallowed>false</plainAuthDisallowed> を指定する（版差に備え <plainAuthEnabled>true</plainAuthEnabled> も併記してよい）。テスト環境のための割り切りであり、本番では TLS を有効化すべき（付録 C）。- 切り分け：メールソフトのエラー表示は情報が乏しいため、PowerShell の TcpClient で IMAP に直接 LOGIN を送って応答を見るのが確実（\$9・\$10）。

POP3 の扱い：本書では **IMAP** のみで十分と判断し、**POP3** は無効化（pop3server.xml の <pop3server enabled="false">）した。開けるポートを減らせる。POP3 も使う場合は \$7.2 のとおり enabled="true" にし、plainAuthDisallowed 相当の設定を確認する。

7.4 メール配送と外部ドメイン宛の送信禁止 (mailetcontainer.xml・2 層防御／案 C)

外部ドメイン宛の送信を禁止する仕組みは、実機検証の結果、**2 層構造**であることが分かった。本書は両層を活かす案 **C** を採用する。

【第 1 層】SMTP の RCPT TO 段階（リレー制御・smtpserver.xml / mailetcontainer.xml の RemoteAddrNotInNetwork)

宛先が外部ドメイン＝「リレー」に該当

→ 送信元 IP が authorizedAddresses / 許可ネットワークに含まれるか判定

含まれない → 【5.7.1 relaying denied】で即拒否（同期エラー）← PLM がその場で失敗を検知できる

含まれる → 受理 (250 OK) して第2層へ
【第2層】mailetcontainerのtransport処理
RemoteDeliveryが存在しない (3.8.2の既定に無い)
→ 外部宛はBounceメイレットに落ちる → 差出人に「送信失敗」が返る

(1) 設定 : 3.8.2の既定にはRemoteDeliveryが無い (削除不要)

3.8.2のmailetcontainer.xmlの既定transportプロセッサには、そもそもRemoteDeliveryが含まれていない。したがって「削除する」作業は不要で、既定のまま外部宛は最終的にBounce + Nullに落ちる。既定の並びは概ね次のとおり (行番号は実機の例) :

```
<processor state="transport" enableJmx="true">
...
<!-- ローカルドメイン宛 (@plm-lab.local) → 配送 -->
<mailet match="RecipientIsLocal" class="LocalDelivery"/>

<!-- リレー制御：許可ネットワーク外からの (外部宛) は relay-denied へ -->
<mailet match="RemoteAddrNotInNetwork=127.0.0.1, 192.168.137.*" class="ToProcessor">
  <processor>relay-denied</processor>
  <notice>550 外部ドメインへの送信は禁止されています (テスト環境のメール送信規則による拒否)</notice>
</mailet>

<!-- ここまでで処理されなかった外部宛 → 送信失敗 (Bounce) -->
<mailet match="All" class="Bounce">
  <message>Transmission to external domains is forbidden.</message>
  <attachment>none</attachment>
  <passThrough>>false</passThrough>
</mailet>
<mailet match="All" class="Null"/>
</processor>
```

(2) 【必須】RemoteAddrNotInNetworkをauthorizedAddressesと揃える

RemoteAddrNotInNetworkの値 (第1層のリレー判定) は、smtpserver.xmlのauthorizedAddressesと同じ範囲に揃える。既定は127.0.0.1だけなので、社内テストのサブネット (192.168.137.*)を追加する。

- 揃えないと、社内ネットワークからの外部宛メールが、意図しない経路 (relay-deniedとBounceの食い違い) になる。

- matcher はカンマ区切り・ワイルドカード表記 (192.168.137.*) を受け付ける (CIDR の authorizedAddresses とは表記が異なる点に注意)。

(3) 案C：送信失敗を「規則による拒否」と明示し、PLM 側で判定する

目的：PLM 開発者が「自分の実装ミスか、サーバの送信規則違反か」を明確に切り分けられるようにする。

- 第1層 (5.7.1 relaying denied) は同期エラーとして返るため、Send-MailMessage や .NET の SmtpClient.Send がその場で例外を投げる。PLM がプログラムで失敗を検知できる (Bounce だけだと SMTP は 250 OK を返すため、プログラムは送信成功と誤認し、後から届くエラーメールを人間が読むまで気づけない)。
- 拒否メッセージ (<notice>) を「規則による拒否」と分かる文言にカスタマイズしておく、例外メッセージにこの文言が出て、実装ミスと誤解されにくい。
- PLM 側の実装で、例外のステータスコードを判定し、ログ／画面表示を切り分ける (実装指針)：

Try

```
smtpClient.Send(message) '送信成功
```

Catch ex As SmtpException

Select Case ex.StatusCode

```
Case SmtpStatusCode.TransactionFailed, SmtpStatusCode.MailboxUnavailable
```

```
'サーバが規則で拒否 (5.7.1 relaying denied / 5.1.1 等)
```

```
Log.Warn($"メールサーバが送信を拒否 (送信規則違反の可能性) : {ex.StatusCode} {ex.Message}")
```

Case Else

```
'接続不可・タイムアウト等 (サーバ未起動・ポート違い)
```

```
Log.Error($"メール送信の接続エラー: {ex.Message}")
```

End Select

End Try

こうすることで、「①実装の問題」「②サーバの規則による拒否」「③接続の問題」を PLM が明示的に切り分けてログ出力できる。**Bounce** (第2層) は万一の保険として残る (第1層を通過してしまった外部宛も、結局は外部に出ず Bounce される)。

(4) なぜ2層とも必要か (セキュリティ)

第1層 (リレー制御) だけでも第2層 (RemoteDelivery 不在) だけでも、外部への送信は防げる。両方あることで、許可ネットワークの内外いずれの送信元でも、外部へは一切出ない (踏み台 = オープンリレーになり得ない)。テスト環境として堅牢な構成。

- ⚠ 実機で確認された挙動：Send-MailMessage (送信元が IPv6 ループバック ::1 で外部宛を送ると、::1 は 127.0.0.0/8 に含まれないため 第1層で 5.7.1 rela

ying denied となる。一方、許可ネットワーク内 (192.168.137.*) からの外部宛は第1層を通過し 第2層の **Bounce** になる。送信元によって「即エラー」か「後から **Bounce**」かが変わるが、いずれも外部には出ない。

8. フェーズ4：起動・サービス化・ファイアウォール

8.1 起動（まずは手動で確認）

bin 配下には次のスクリプトがある (3.8.2 実機で確認) : **run.bat** (コンソール/フォアグラウンド起動)、**james.bat** (Windows サービス管理)、**james-cli.bat** (CLI・ポート 9999)、**setenv.bat** (JVM オプション)、**wrapper-windows-x86-64.exe** (Tanuki Service Wrapper 本体)。

まずはコンソールで起動し、ログを見ながら確認する。

```
cd /d C:\opt\james\bin
run.bat
```

- run.bat はフォアグラウンドで動き続ける (停止は Ctrl+C)。ログが画面に流れる。
- 起動完了ログ : **Apache James Server is successfully started in ... milliseconds.**
- **SMTP Service bound to: 0.0.0.0:25 / IMAP Service bound to: 0.0.0.0:143** が出れば各プロトコルは待受開始。
- LDAP リポジトリは起動時に JMX 登録される (name=usersrepository)。ただし **bind** 認証の成否は **IMAP** ログイン時に初めて分かる (起動時は接続のみ)。

8.2 待ち受け確認

```
netstat -ano | findstr ":25 "
netstat -ano | findstr ":143 "
netstat -ano | findstr ":110 "
```

25 と **143** が **LISTENING** であること。**110 (POP3)** は無効化したので出ない (本書は **IMAP** のみ。\$7.3)。

8.3 ファイアウォール (受信許可)

バッチサーバ・クライアント PC から接続するため、受信を許可する (管理者 PowerShell)。

```
New-NetFirewallRule -DisplayName "Allow SMTP 25 (James)" -Direction Inbound -Protocol TCP -LocalPort 25 -Action Allow
# POP3 を使う場合のみ : New-NetFirewallRule -DisplayName "Allow POP3 110 (James)" -Direction Inbound -Protocol TCP -LocalPort 110 -Action Allow
New-NetFirewallRule -DisplayName "Allow IMAP 143 (James)" -Direction Inbound -Protocol TCP -LocalPort 143 -Action Allow
```

8.4 Windows サービス化 (james.bat ・ NSSM 不要)

動作確認 (\$9) が済んだら、James を Windows サービスとして登録し、サーバ起動時に自動起動させる。3.8.2 は **james.bat** (**Tanuki Service Wrapper** 同梱) でサービス化でき、**NSSM** 等の外部ツールは不要。

動作確認の前でも登録は可能だが、まず run.bat (コンソール) で設定の正しさを確認してから、サービス化するのが安全。サービス化する前に、**run.bat** を起動しているコンソールは **Ctrl+C** で停止しておく (同じポートを二重に掴まないため)。

(1) james.bat のサブコマンドを確認

管理者コマンドプロンプトで、引数なしで実行すると使用法が表示される。

```
cd /d C:\opt\james\bin
james.bat
```

一般に次のサブコマンドを持つ (実機の表示で確認する) : install (サービス登録) / start (開始) / stop (停止) / restart / remove (登録解除) / console (コンソール実行)。

(2) サービスの登録と開始

管理者権限のコマンドプロンプト (PowerShell ではなくコマンドプロンプト推奨) で実行する。

```
cd /d C:\opt\james\bin
james.bat install    :: Windows サービスとして登録
james.bat start      :: サービスを開始
```

- install 成功後、サービス一覧に James のサービスが追加される。実機での表示名 (**DisplayName**) は「**Apache James :: Server :: Spring :: App**」 (wrapper.conf の定義による)。
- start 後、\$8.2 と同様に待ち受けを確認する。

```
netstat -ano | findstr ":25 "
netstat -ano | findstr ":143 "
```

(3) スタートアップの種類を「自動」にする

サーバ再起動後も自動復旧させるため、スタートアップの種類を確認・設定する。

サービス名の確認 (*DisplayName に James を含むものを探す*)

```
Get-Service | Where-Object { $_.DisplayName -like "*James*" } |
Select-Object Name, DisplayName, Status, StartType
```

→ *DisplayName* 「**Apache James :: Server :: Spring :: App**」 が該当 (実機で確認)

スタートアップを自動に (<ServiceName> は上で確認した Name)

```
Set-Service -Name "<ServiceName>" -StartupType Automatic
```

DisplayName に記号 (::) が含まれるため、Get-Service で指定する際は **Name** (内部のサービス名) を使うのが確実。Get-Service | Where-Object { \$_.DisplayName -like "*James*" } で該当行の Name を確認し、以降のコマンドではその Name を用いる。

services.msc (サービス管理) から GUI で「スタートアップの種類：自動」に設定してもよい。ApacheDS・Tomcat (SSO 用) と揃える。

(4) サービス動作の確認

サービスの状態

```
Get-Service -Name "<ServiceName>" | Select-Object Name, Status, StartType
```

- Status = Running / StartType = Automatic であること。
- サービスとして動かすと run.bat のコンソールは不要。ログは **C:\opt\james\log\配下** (wrapper.log / james-server.log 等) に出力される。

(5) 再起動堅牢性の確認 (§9.4 の項目 8)

仮想マシンを再起動し、ログインなしで (コンソールを開かずに) 次を確認する。

```
Get-Service -Name "<ServiceName>" | Select-Object Status
```

```
netstat -ano | findstr ":25 "
```

```
netstat -ano | findstr ":143 "
```

再起動後に自動で Running・25/143 が LISTENING になっていれば、サービス化は成功。§9.2 の送受信テストを再実行して、通常どおり動作することを確認する。

依存関係の注意：James は **ApacheDS (LDAP)** が起動していないと認証できない。両方を「自動」にしておけば通常は問題ないが、起動順序で James が先に立ち上がって LDAP 未起動の一瞬があっても、James は接続をリトライするため実害は出にくい。気になる場合は、サービスの依存関係 (James が ApacheDS に依存) を設定する。

任意：James が ApacheDS より後に起動するよう依存関係を設定 (サービス名は実機で確認)

```
sc.exe config "<JamesServiceName>" depend= "<ApacheDSServiceName>"
```

(6) 停止・登録解除 (メンテナンス時)

設定変更などでサービスを外す場合：

```
cd /d C:\opt\james\bin
```

```
james.bat stop    :: 停止
```

```
james.bat remove  :: 登録解除
```

設定ファイル (conf*.xml) を変更したときは、james.bat restart (または stop-start) で反映する。

9. フェーズ 5 : 動作確認

9.1 SMTP の疎通 (PowerShell)

この仮想マシンには **telnet** が無い場合が多いため、PowerShell の Send-MailMessage を主手段とする。

ユーザー間 (小文字で指定) — 成功すると何も表示されない

```
Send-MailMessage -SmtpServer localhost -Port 25 `
-From "01plm02@plm-lab.local" -To "01plm01@plm-lab.local" `
-Subject "test mail" -Body "this is a test." -Encoding UTF8
```

- エラーが出なければ SMTP 受付 + ローカル配送は成功。James のコンソールに **Local delivered mail ... in folder INBOX** が出れば配送確定。
- 宛先・差出人は小文字 (01plm01@...) で指定する (James は内部で小文字正規化するため。\$9.5)。大文字でも James が正規化して届くが、表記を実体に合わせるため小文字を用いる。

SMTP の応答コードを直接見たい場合は、TcpClient で対話できる (telnet 相当) :

```
$c = New-Object System.Net.Sockets.TcpClient("localhost",25); $s=$c.GetStream()
$r=New-Object IO.StreamReader($s); $w=New-Object IO.StreamWriter($s); $w.AutoFlush=$true
Start-Sleep -m 300; $r.ReadLine()
function Cmd($x){ $w.WriteLine($x); Start-Sleep -m 300; while($s.DataAvailable){$r.ReadLine()}}
Cmd "EHLO test"
Cmd "MAIL FROM:<01plm02@plm-lab.local>"
Cmd "RCPT TO:<01plm01@plm-lab.local>"
Cmd "DATA"; $w.WriteLine("Subject: test"); $w.WriteLine(""); $w.WriteLine("body"); Cmd ".
"
Cmd "QUIT"; $c.Close()
```

250 2.6.0 Message received 相当が返れば受理されている。

telnet を使いたい場合のみ : dism /online /Enable-Feature /FeatureName:Telnet Client

9.2 メールソフトによる送受信テスト（ユーザー間）

PLM を絡めずに、メールサーバ単体の機能（SMTP 受付 - ローカル配送 - IMAP/POP 3 取得）を検証する。切り分けとして有効なので、PLM 連携（\$9.3）の前に必ず実施する。

(1) クライアント PC のメールソフトに 2 アカウントを設定

Thunderbird 等に、送信側と受信側の 2 つのアカウントを追加する。

項目	アカウント 1	アカウント 2
メールアドレス	01plm01@plm-lab.local	01plm02@plm-lab.local
ユーザー名 (ログイン ID)	01plm01@plm-lab.local (小文字・メールアドレス全体)	01plm02@plm-lab.local
パスワード	01PLM01 (ApacheDS のパスワード。大文字のまま)	01PLM02
受信サーバ (IMAP)	メールサーバの IP/ホスト名 / ポート 143 / 接続の保護：なし / 認証：通常のパスワード (安全でない)	同左
送信サーバ (SMTP)	同上 / ポート 25 / 接続の保護：なし / 認証：なし	同左

必ず「手動設定」を使う：自動設定（アカウント自動検出）は暗号化を前提とするため失敗する。接続の保護 = なし、認証方式 = 通常のパスワード（安全でない）、送信の認証 = 認証なしを明示的に選ぶ。暗号化なしの警告が出るが、テスト環境のため許容する。

⚠ **IMAP** でパスワード入力画面すら出ずにつながらない場合：imapserver.xml の **plainAuthDisallowed=false**（\$7.3）が未設定だと、平文認証が拒否され、メールソフトは認証に進まず即切断する。\$7.3 を確認する。

ログイン ID・パスワードの大文字小文字：ログイン ID は小文字のメールアドレス全体（01plm01@plm-lab.local）。パスワードは LDAP 登録値（01PLM01）で大文字のまま（パスワードは正規化されない）。

(2) ユーザー間のテストメール送受信

1. アカウント 1（01plm01@plm-lab.local）から、宛先 **01plm02@plm-lab.local** でメールを送信する。
2. James が受け取り、RecipientIsLocal にマッチ - LocalDelivery で 01plm02 のメールボックスへ配送（\$7.4）。
3. アカウント 2（01plm02@plm-lab.local）で受信し、メールが届いていることを確認する。
4. 逆方向でも同様に確認する。

これが成功すれば、**LDAP 認証 (ApacheDS への bind)** ・ **SMTP 受付** ・ **ローカル配送** ・ **IMAP 取得の一連**がすべて機能している (IMAP ログインで初めて LDAP の bind 認証が行われる)。

(3) 外部ドメイン宛が「送信失敗」になることの確認 (§7.4 の2 層防御)

localhost (:::1) から外部宛 → 第1層で 5.7.1 relaying denied (即エラー)

Send-MailMessage -SmtpServer localhost -Port 25 `

-From "01plm01@plm-lab.local" -To "test@example.com" `

-Subject "external test" -Body "should be blocked." -Encoding UTF8

- **5.7.1 Requested action not taken: relaying denied** の例外が返れば、第1層 (リレー拒否) が効いている (PLM がその場で失敗を検知できる = 案 C)。
- 許可ネットワーク内 (192.168.137.*) から同様に送ると、第1層を通過し第2層の **Bounce** (差出人にエラーメール) になる。送信元によって挙動が変わるが、いずれも外部には出ない (§7.4(4))。

(4) メールアドレスの大文字小文字 (重要)

- **James** はメールアドレスを内部で小文字に正規化する (james-cli listusers は 01plm01@plm-lab.local と表示)。大文字宛 (01PLM01@...) でも James が正規化するため届くが、運用は小文字で統一するのが素直 (設定と実体の表記が一致し、トラブル時に混乱しない)。
- **LDAP** の **mail/SSO** の **REMOTE_USER** は大文字 (01PLM01@...) のまま維持する。小文字に変えると SSO の REMOTE_USER も小文字になり、PLM の識別番号 (01PLM01) と不一致になる恐れがある。
- **PLM** 実装: メール宛先を組み立てる際に ToLower() で小文字化する (付録 B)。

(5) ログインできない場合の切り分け

- ログイン ID は小文字のメールアドレス全体 (01plm01@plm-lab.local)。01plm01 だけでは失敗する。
- パスワード入力画面すら出ない → plainAuthDisallowed=false (§7.3)。
- Unknown user になる → supportsVirtualHosting (§6.1)。まず **james-cli.bat -h localhost -p 9999 listusers** で James が認識しているユーザーを確認する (§10)。

9.3 PLM (Web アプリ/バッチ) からの送信

PLM 側のメール送信設定を、この James に向ける。

設定項目	値
SMTP サーバ	メールサーバのホスト名/IP (Web アプリサーバ自身。バッチサーバからはその IP)
SMTP ポート	25

設定項目	値
認証	なし (authorizedAddresses でリレー許可済み)
暗号化	なし
差出人	例: plm-system@plm-lab.local (ローカルドメイン内のアドレス・小文字)
宛先	@plm-lab.local の小文字アドレス (例 01plm01@plm-lab.local) – James 内のメールボックスに配送される。PLM 側は ToLower() で小文字化
.NET (PLM が .NET Framework の場合) の例: System.Net.Mail.SmtpClient の Host/Port=25/EnableSsl=false/認証なし。 - 送信失敗の切り分け (案 C) : 外部宛や規則違反は SmtpException として同期的に返るので、ステータスコードで「規則による拒否」「接続エラー」「実装エラー」を切り分けてログ出力する (§7.4(3) の実装例)。これにより開発者が「自分の実装ミスか、サーバの規則違反か」を明確に判断できる。 - PLM が送信したメールを、§9.2 のメールソフトで受信できれば、要件 (PLM のメール送信結果の確認) が達成される。	

バッチサーバからの送信が拒否される場合: smtpserver.xml の <authorizedAddresses> と mailletcontainer.xml の RemoteAddrNotInNetwork にバッチサーバの IP/サブネットが含まれているかを確認する (§7.1・§7.4(2))。

9.4 動作確認チェックリスト

#	確認内容	期待結果
1	James 起動	25/143 が LISTENING (110 は POP3 無効のため出ない)
2	LDAP 連携	起動ログに LDAP 接続エラーが無い
3	SMTP 受付	Send-MailMessage (小文字宛) がエラーなし。ログに Local delivered ... INBOX
4	IMAP 受信	メールソフトで 01plm01@plm-lab.local にログインでき、メールが見える (= LDAP bind 認証成功)
4b	ユーザー間送受信	01PLM01 – 01PLM02 のメールが届く (メールサーバ単体の検証。§9.2 (2))

#	確認内容	期待結果
4c	外部宛の送信禁止	外部ドメイン宛（例 test@example.com）が送信失敗になり、Bounce が返る (§9.2(3))
6	Web アプリからの送信	PLM が送ったメールが受信できる
7	バッチサーバからの送信	別ホストからのリレーが許可され、メールが受信できる
8	再起動堅牢性	サーバ再起動後（サービス化後）、James が自動起動して同様に動作する

9.5 正常時にも出る紛らわしいログ (判断基準)

James のログは「ERROR という文字列」ではなく内容で判断する（SSO 手順書と同じ考え方）。

ログ	判断
ERROR ... ToSenderFolder ... in folder Sent	無害。送信者の Sent フォルダへ保存した記録（ログレベル指定の癖で ERROR 表記だが正常）
INFO ... Can not locate SIEVE script for user ...	無害。Sieve（振り分けルール）未使用
WARN ... No authentication setted up for the JMX component	無害。ローカル管理用 JMX。テスト環境では許容
WARN ... openjpa ... ClassTransformer	無害。James が消費済み
WARN ... ActiveMQ ... resetting to ... (memory/store)	無害。上限を JVM/ディスクに合わせ自動調整
INFO ... Local delivered mail ... in folder INBOX	★配送成功。これが出れば OK
Rejected message. Unknown user: ...	要対処。supportsVirtualHosting/LDAP を確認 (§6.1・§10)
NO LOGIN failed. Plain login / authentication are disabled.	要対処。plainAuthDisallowed=false (§7.3)
Exception + スタックトレースを伴う ERROR	要対処

10. トラブルシューティング

症状	主な原因	対処
起動時 UnsupportedC	James 3.9 は Java 21 必須。Java 17 では動かない	James 3.8.2 を使う (§5.1)。65.0=Java21/6

症状	主な原因	対処
lassVersionError (class file 65.0 vs 61.0)		1.0=Java17
SMTP で 5.1.1 Unknown user (LDAP・ドメインは正しいのに)	enableVirtualHosting は無視される (タグ名違い)	supportsVirtualHosting を使う (§6.1)。 まず james-cli.bat -h localhost -p 9999 listusers で James が認識するユーザーを確認
IMAP でパスワード入力画面すら出ず接続断	暗号化なしで平文認証が拒否されている	imapserver.xml に plainAuthDisallowed=false (§7.3)。PowerShell の TcpClient で LOGIN を送ると NO ... Plain login ... disabled が見える
差出人 @plm-lab.local のメールが拒否される	verifyIdentity=true (既定)	smtpserver.xml で verifyIdentity=false (§7.1・認証なし運用のため)
james-cli listusers が空	LDAP リポジトリが読み込まれず JPA が使われている	usersrepository.xml の LDAP 定義が有効か・JPA 定義がコメントアウトされているか。conf 配下に余計な .xml コピーが無いか (.xml.org にする)
起動時に LDAP エラー	ldapHost／principal／credentials／userBase の誤り	§6.1 の値を確認。ApacheDS が起動し 10389 が LISTENING か
ログインできるがメールが届かない	ドメインが未登録	domainlist.xml に plm-lab.local (§6.2)。無いと外部扱いで Bounce される
PLM／バッチからの外部宛が拒否	案 C の意図どおり (外部送信禁止)	宛先が @plm-lab.local か確認。外部宛は仕様上禁止 (§7.4)
バッチからのローカル宛が拒否	リレー判定の範囲不足	authorizedAddresses (§7.1) と RemoteAd

症状	主な原因	対処
他ホストから接続できない	バインド／ファイアウォール	drNotInNetwork (§7.4 (2)) に送信元サブネットを追加 <bind> が 0.0.0.0 か (§7)。ファイアウォールで 25/143 を許可 (§8.3)
メールソフトの自動設定が失敗	自動検出は暗号化前提	手動設定で「接続の保護：なし」「認証：通常のパスワード（安全でない）」「送信認証：なし」 (§9.2)
大文字宛で Unknown user (supportsVirtualHosting 設定後)	—	設定後は大文字でも届く (James が小文字正規化)。運用は小文字に統一 (§9.2 (4))

ログの確認：C:\opt\james\log\ 配下。判断基準は §9.5。真因が不明な LDAP 問題は、log4j2.xml に <Logger name="org.apache.james.user.ldap" level="debug"> を追加して再起動すると詳細が出る。

付録 A：ユーザーの追加

ユーザーは **ApacheDS** 側で管理する (James は読み取り専用)。新しいユーザーを追加するには、SSO 構築手順書のフェーズ 3 と同じ要領で、Apache Directory Studio から LDI F を投入する。

```
dn: uid=01PLM03,ou=people,dc=example,dc=com
objectClass: inetOrgPerson
objectClass: organizationalPerson
objectClass: person
objectClass: top
uid: 01PLM03
cn: 01PLM03
sn: PLM
mail: 01PLM03@plm-lab.local
userPassword: 01PLM03
```

- 投入後、**James** の再起動は不要 (LDAP を都度参照するため)。

- mail 属性のドメインは、必ず **plm-lab.local** (domainlist.xml に登録したローカルドメイン) にする。
- mail の値は **SSO** と揃えて大文字 (**01PLM03@plm-lab.local**) のままでよい。James が内部で小文字に正規化するため、メール送信時・IMAP ログイン時は小文字 (01plm03@plm-lab.local) を用いる (付録 B)。

付録 B : SSO 環境との関係

項目	SSO (Shibboleth)	メール (James)
LDAP	ApacheDS (同一)	ApacheDS (同一)
ユーザー	uid=01PLM01	同じエントリ
識別子	NameID = mail (01PLM01@plm-lab.local)	ログイン ID = mail (同じ値)
認証	IdP が LDAP に bind	James が LDAP に bind
大文字小文字	大文字 (01PLM01@...。REMOTE_USER にそのまま渡す)	小文字 (01plm01@...。James が正規化)

ユーザー管理が **ApacheDS** に一元化されるため、SSO のテストユーザーがそのままメールアカウントになる。ユーザーを増やす際も、LDAP に 1 回追加するだけで両方に反映される。

大文字小文字の使い分け (重要) : LDAP の mail と SSO の REMOTE_USER は大文字 (01PLM01@plm-lab.local)。PLM はこの @ の前 (01PLM01) を識別番号として認可判定する。一方、**James** はメールアドレスを小文字に正規化するため、メール送信の宛先・IMAP ログイン ID は小文字 (01plm01@plm-lab.local) を用いる。**LDAP** の **mail** を小文字に変えてはいけない (SSO の REMOTE_USER も小文字になり、PLM の識別番号と不整合になる)。PLM 実装では、メール宛先を組み立てる際に ToLower() で小文字化する :

```
Dim userId As String = "01PLM01" '認可判定用 (大文字・DB 照合)
Dim mailTo As String = (userId & "@plm-lab.local").ToLower() 'メール宛先 (小文字)
```

SSO (大文字) とメール (小文字) で表記が異なるが、両者は別の処理系で直接データを渡し合わないため実害はない。共通しているのは「同じ LDAP ユーザーを見る」ことだけ。

付録 C : 本番・組織展開に向けた留意点

本書はテスト環境向けに暗号化なし・認証なし (**IP** リレー許可) で構成している。組織の環境やポリシーによっては、次の検討が必要になる。

- 暗号化 : STARTTLS (SMTP 587 / IMAP 143 + STARTTLS) または SSL (465 / 993 / 995) の有効化。James は keystore を設定して対応できる。
- **SMTP AUTH** : IP ベースのリレー許可ではなく、送信時に認証を要求する。

- リレー範囲：authorizedAddresses は必要最小限に絞る（広いネットワークを許可すると踏み台になり得る）。
- 外部メール送信：本書はテスト環境内で完結させる構成。実際に社内メールサーバへ中継する場合は、RemoteDelivery のゲートウェイ設定（スマートホスト）を用いる。
- 上記はいずれも組織のセキュリティポリシーに依存するため、ネットワーク管理者と相談のうえ決定すること。